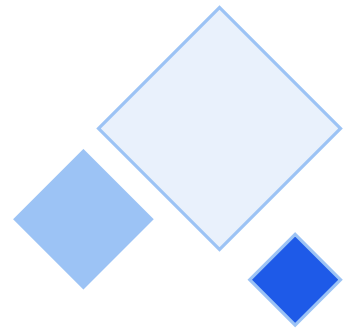




THE KYA METHOD

How to build AI systems you can trust.

Promise it. Attack it. Inspect it. Prove it.

Rume Dominic

Creator of the KYA Framework · UK patent pending · rumedominic.com

Creating is my trade; teaching is what creating obliges.

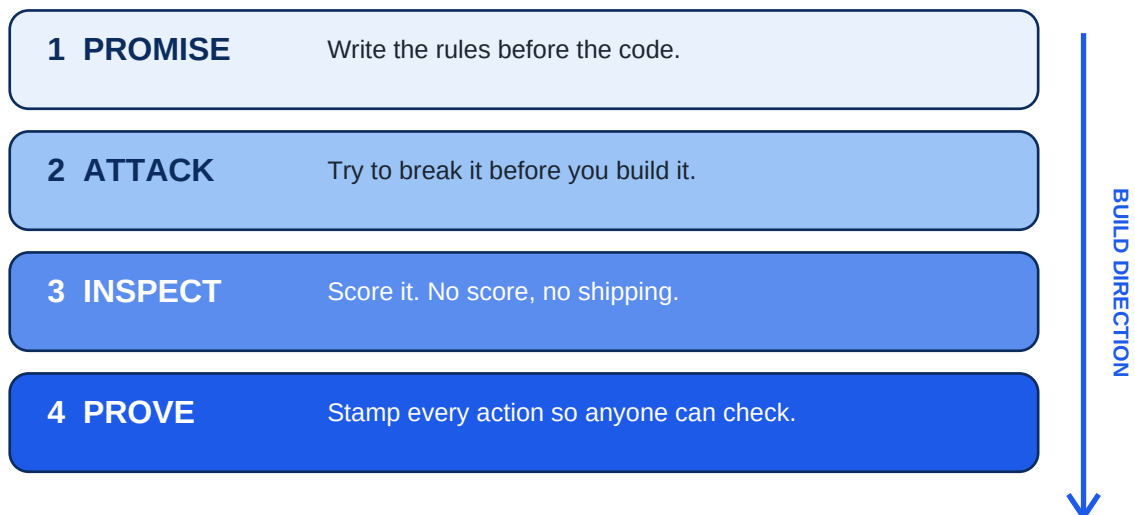


The Big Idea

Two AI systems can look exactly the same in a demo. Both run. Both impress. But one falls apart the first time something real goes wrong. The other stands.

Why? Because the difference is never in the demo. It is in the discipline underneath.

The KYA Method is that discipline. It is four steps. Do them in order, every time, and you get software that survives the real world — and can prove what it did.



One rule holds the whole method together: the AI suggests, the human decides. Your AI tools can propose a thousand ideas. Only you decide what becomes real. Nothing ships because it looks right. Things ship because they passed.



Step 1 — Promise

Write the rules before the code.

Before you build anything, you write down the promise: what this system must always do, what it must never do, and what finished means. You write it in plain files, in plain words, kept right inside the project folders.

The folders themselves are the plan. Each stage of the work gets its own folder. Each folder holds one short file that says: here is what comes in, here is what you do, here is what goes out, here is how you know you are done. Short is the rule — if you cannot say it in one page, you have not understood it yet.

There is also a second list, and it matters just as much: what NOT to look at. An AI given everything gets lost in everything. An AI given only what the step needs works fast and stays sharp. Focus is designed, not hoped for.

The test for Step 1: could a stranger open your folders, read the files, and know exactly what the system promises — without asking you a single question? If yes, you have a Promise. If no, you have a wish.



Step 2 — Attack

Try to break it before you build it.

A design everyone agrees with is a design nobody has tested. So before building, you attack your own plan from five directions. Same AI, five different hats:



Here is the secret: you are not collecting answers. You are collecting arguments. Where the five hats disagree — that is exactly where your design is weakest. Fix it on paper, where fixing costs minutes, not in production, where fixing costs customers.

Then build in small pieces. Each piece gets three tests: Does it work? Does it connect to the rest? And the one most people skip — what happens when things go wrong? Cut the network. Feed it bad data. Double the load. Watch what breaks. Better you break it on a Tuesday than a customer breaks it on launch day.

And every shortcut you take, you write down. A written shortcut is a debt you will pay. An unwritten shortcut is a trap you will step in.



Step 3 — Inspect

Score it. No score, no shipping.

Before anything ships, it faces an inspection — thirteen checks that ask one question thirteen ways: is this a real system, or just a good-looking demo? Are the rules written down? Was the plan made before the code? Can one failure spread? Can you see how the AI reached its answer? Does it check its own work? Does it ship through a pipeline or by hand? Does it learn from its last run?

Three honest grades, nothing in between:

PASS	It is built, and you can point to the exact file that proves it.
GROWING	It is started but not finished. Name what is missing.
MISSING	It is not there. Fix it, or sign your name to why it can wait.

Two things make this inspection different from a checklist on a wall. First: pointing is required. You cannot say "we handle that." You must point to the file, the test, the mechanism. Talk is not evidence. Second: the inspection knows YOU. Every builder has favourite corners to cut — the method keeps your personal list, and those checks cannot be waved through. Your blind spots are written down so they stop being blind.

And the gate is a program, not a promise. A script runs the inspection. If the score is not there, the ship button does not work. Discipline you cannot forget is the only discipline that lasts.



Step 4 — Prove

Stamp every action so anyone can check.

This is the step the rest of the industry skips, and the reason this method carries the KYA name — Know Your AI.

Every important thing the system does gets a stamp: what happened, when, under which rule, approved by whom. Each stamp is sealed with mathematics, and each seal locks the one before it — like a chain where changing any link breaks every link after it. Touch the record, and the record shows the touch.



What this buys you is the rarest thing in AI today: a system whose word you do not have to take. A client asks "did the AI really do what you say?" — you do not answer with trust me. You hand them the chain and they check it themselves. Not a lock — a witness.

Steps 1 to 3 build software that survives. Step 4 builds software that can be believed. In a market drowning in demos, believable wins.



The Loop, and Where This Comes From

The four steps are not a ladder you climb once. They are a wheel you turn every cycle: Promise, Attack, Build and test, Inspect, Prove — ship — then turn the wheel again. Every turn makes the rules sharper, pays down a debt, closes a gap, and adds links to the chain. Speed comes from the discipline, not despite it.

Who this is for

Any builder shipping AI into the real world: a founder with one product, a team inside an enterprise, a student turning research into tools. The method does not care how big you are. It cares whether your system can take a punch and prove its story afterwards.

Standing on named shoulders

Honest builders credit their materials. The folder-first way of guiding AI builds on the Interpretable Context Methodology by Jake Van Clief and David McDermott. The thirteen-check inspection grew from the system-audit thinking of Gabriel Millien, with visual lineage to Brij Kishore Pandey. The attack-first engineering loop belongs to the wider craft of systems engineering. What is mine is the fourth step — the proof layer, from my KYA Framework (UK patent pending) — and the welding of all four into one method a builder can run every day.

The whole method in four lines:
Write the promise before the code.
Attack the plan before the build.
Inspect the work before the ship.
Stamp the actions so anyone can check.

Rume Dominic is an AI and blockchain engineer, founder of Vorem, author of five books, and creator of the KYA Framework for verifiable AI accountability. He has trained 12,000+ learners across four continents. · rumedominic.com